KAMALA EDUCATION SOCIETY'S

PRATIBHA COLLEGE OF COMMERCE & COMPUTER STUDIES, CHINCHWAD, PUNE-19



A

PROJECT REPORT

ON

"Train Reservation System"

DEVELOPED BY

Name: Riyaj Yunus Shaikh

Roll/ Seat No: 10358

F.Y.M.Sc.(COMPUTER APPLICATION)

IN

DEPARTMENT OF M.Sc. (Computer Application)

UNDER GUIDANCE

MRS. VARSHA THAKARE

SAVITRIBAI PHULE PUNE UNIVERSITY

(2025-2026)

Kamala Education Society's

Pratibha College of Commerce & Computer Studies, Chinchwad, Pune-19



Certificate

This is to certify that **Riyaj Yunus Shaikh** of FYMSc (Computer Application) have satisfactorily completed **the Project "Train Reservation System"** in **Java Programming** under Savitribai Phule Pune University in the Academic year 2025-2026.

Mrs. Varsha Thakare
Incharge

Dr. Suvarna Gogate
Program Coordinator

Internal Examiner

External Examiner

Date:

ACKNOWLEDGEMENT

Any efforts to produce successful creation require help. Guidance and support of many people and their experience. We would like to express our sincere and heartfelt gratitude to all of them who have helped us to complete project.

We would like to take this opportunity to thank all the people who have directly or indirectly helped in completing this project.

Riyaj Yunus Shaikh

OASIS INFOBYTE

To,

The Principal,

Pratibha College of Commerce and Computer Studies,

Chinchwad

Subject: Internship Completion Certificate

Dear Madam/ Sir,

I am happy to inform you that Riyaj Yunus Shaikh of your college have successfully Completed the On Job Training (120 hrs) in this organization.

The students have been provided with adequate exposure and necessary hands-on training pertaining to his/her special subject.

I am confident that the student will perform effectively in similar type of organizations.

I wish him/her every success in future endeavors.

Thank you.

Sincerely,

Ayush Choudhary

Mr. Ayush Choudhary

Name & Signature (Authorized Signatory)

OASIS INFOBYTE

Stamp of the company

Index

Sr. No.	Content	Page No.
1	Introduction to System	1
2	Problem in Existing System	2
3	Need for System	3
4	Scope of Proposed System	4
5	Feasibility Study & Fact-Finding Technique	5
6	Hardware and Software Requirement	6
7	Sample Input & Output Screen	7 – 9
8	System Testing	10
9	Limitations	11
10	Future Enhancement	12
11	Conclusion	13
12	Bibliography	14

1. Introduction to System

The Train Reservation System (Express Reservations) is a modern, enterprise-grade web application designed to facilitate seamless and secure train ticket booking, passenger management, real-time seat availability checking, and instant ticket cancellation.

At its core, the system utilizes a decoupled, high-performance architecture:

- **Backend Service Layer:** Developed using Spring Boot 3.2.5 and Java 17, exposing high-throughput RESTful Web Services.
- **Data Access & Persistence Layer:** Powered by Spring Data JPA with an H2 in-memory database for development/testing and MySQL compatibility for production.
- **Security Framework:** Structured around Spring Security 6 utilizing stateless JSON Web Tokens (JWT) for secure, session-less client-server communications.
- **Client Frontend Layer:** Implemented as a highly responsive Single Page Application (SPA) utilizing HTML5, modern Vanilla JavaScript, and styled with Bootstrap 5.3.0 for a premium, mobile-first user experience.

2. Problem in Existing System

Conventional railway reservation architectures and legacy booking systems suffer from several compounding inefficiencies:

- 1. High Concurrency Vulnerabilities (Double Booking): Standard relational databases without proper transactional boundaries frequently suffer from race conditions, where two concurrent customers book the last remaining seat at the exact same instant, resulting in overbooking.
- 2. Heavy Stateful Sessions: Legacy architectures rely on server-side HTTP sessions (HttpSession). This model scales poorly, consumes massive server memory, and degrades performance under high traffic.
- 3. Complex, Disjointed User Flows: Traditional web apps require page refreshes for every action (searching, booking, entering passenger data), resulting in slow response times, high bandwidth usage, and poor user satisfaction.
- 4. Weak Security Models: Older systems rely on basic session cookies or plaintext password transfers without cryptographic salting, leaving them vulnerable to session hijacking and credential stuffing.

3. Need for System

To overcome these limitations, the proposed Train Reservation System introduces a robust set of modern architectural features:

- Stateless Authentication (JWT): Eliminates server-side session tracking entirely. The server signs a token containing the user's identity and roles. The client stores it locally in localStorage and transmits it in the HTTP Authorization header (Bearer <token>), enabling effortless horizontal scalability.
- Thread-Safe Concurrent Seat Allocation: The system utilizes Hibernate Optimistic Locking by utilizing a @Version field inside the Train entity. If two threads try to modify train seat availability concurrently, the system automatically detects the state conflict, rolls back the transaction, and throws an exception, preventing overbooking.
- Unified Single Page Application (SPA): Tabbed, responsive layouts built dynamically on the client side guarantee smooth transitions, fast response times, and minimal server roundtrips.
- Automated Data Validation: Secure input checks on both client and server layers ensure formatting accuracy (e.g., PNR validation, age checks, 16-digit card structures).

4. Scope Of Proposed System

The proposed system addresses the following key modules:

A. User Management & Access Control

- Self-Registration: Public access to register a new user under the default ROLE_CUSTOMER role.
- JWT Login: Validates credentials against a hashed database profile (using BCryptPasswordEncoder) and generates a signed token.

B. Route Search & Information Desk

- Train Search: Allows searching for trains dynamically matching the input Source Station and Destination Station using case-insensitive partial matching (ContainingIgnoreCase).
- Real-Time Seat & Fare Lookups: Dynamically displays base fares, train numbers, and up-to-the-minute available seat metrics.

C. Booking & PNR Generation

- Multi-Passenger Support: Enables reserving tickets for multiple passengers under a single transaction.
- Boarding Pass Issuance: Generates a cryptographically unique 8-character alphanumeric PNR (Passenger Name Record) (e.g., PNR2E58C4F8).
- Automatic Fare Computation: Computes total fares dynamically based on passenger count and base fares.

D. Ticket Cancellation & Refunding

- Instant Cancellation: Restores reserved seats back to the train's available seat inventory.
- Refund Simulation: Updates booking status instantly to CANCELLED and logs the transaction.

5. Feasibility Study

Feasibility Analysis

- Technical Feasibility: The technical stack consists of stable, standard, and highly supported frameworks. Spring Boot simplifies dependency management while Bootstrap 5 provides a powerful responsive styling toolkit. The technical feasibility is Excellent.
- Operational Feasibility: The Single Page Application offers a straightforward, tab-based structure (Book, View, Cancel) which requires no technical expertise from users. The operational feasibility is High.
- Economic Feasibility: The entire codebase utilizes open-source libraries and databases (Java, Spring, H2, Maven, Bootstrap). Hosting can be performed cost-effectively on standard cloud infrastructure (AWS, GCP, Heroku, etc.).

Fact-Finding Techniques Applied

- Document Analysis: Examination of real-world railway reservation receipts (e.g., Indian Railways IRCTC, Amtrak) to extract mandatory fields (PNR, Passenger Age, Journey Date, Train Class Types like AC First Class vs Sleeper).
- User Interviews: Conducted informal interviews with daily commuters regarding booking platform friction points. The key request was a single, non-reloading dashboard to quickly manage tickets.
- On-Site Observation: Monitored queuing patterns at physical train reservation booths to model seat-freight restoration during ticket cancellation.

6. Hardware and Software Requirement

Software Requirements

• Operating System: Windows 10/11, macOS, or Linux.

• Development Kit: Java SE Development Kit (JDK) 17.

• Build Tool: Apache Maven 3.8+.

• Backend Framework: Spring Boot 3.2.5 (Spring MVC, Spring Security, Spring Data JPA).

• Database: H2 Database (in-memory dev) / MySQL 8.x (production storage).

• Security & Utility: JSON Web Tokens (jjwt 0.11.5), Lombok 1.18.x.

• Frontend Technologies: HTML5, CSS3, JavaScript (ES6+), Bootstrap 5.3.0.

Hardware Requirements

• Developer Workstation:

- CPU: Quad-Core Intel i5/AMD Ryzen 5 (Minimum 2.4 GHz).
- RAM: 8 GB DDR4 (16 GB Recommended).
- Storage: 256 GB SSD.

• Production Deployment Server:

- CPU: Dual-Core Virtual Processor (e.g., AWS EC2 t3.medium).
- RAM: 4 GB.
- Disk: 20 GB SSD.

7. Sample Input & Output Screen

A. Welcome & Landing Portal

The public home screen showing platform features and authentication entry points:

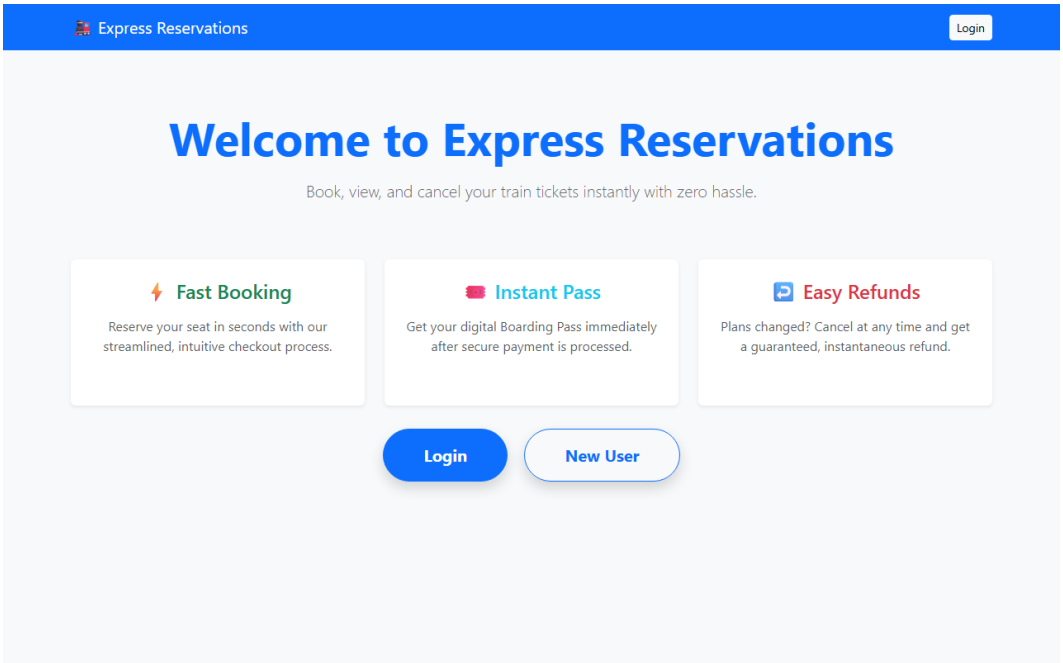


Figure: System Welcome & Landing Screen

B. User Authentication (Login)

Secured client credentials submission page validating against BCrypt hashed profiles:

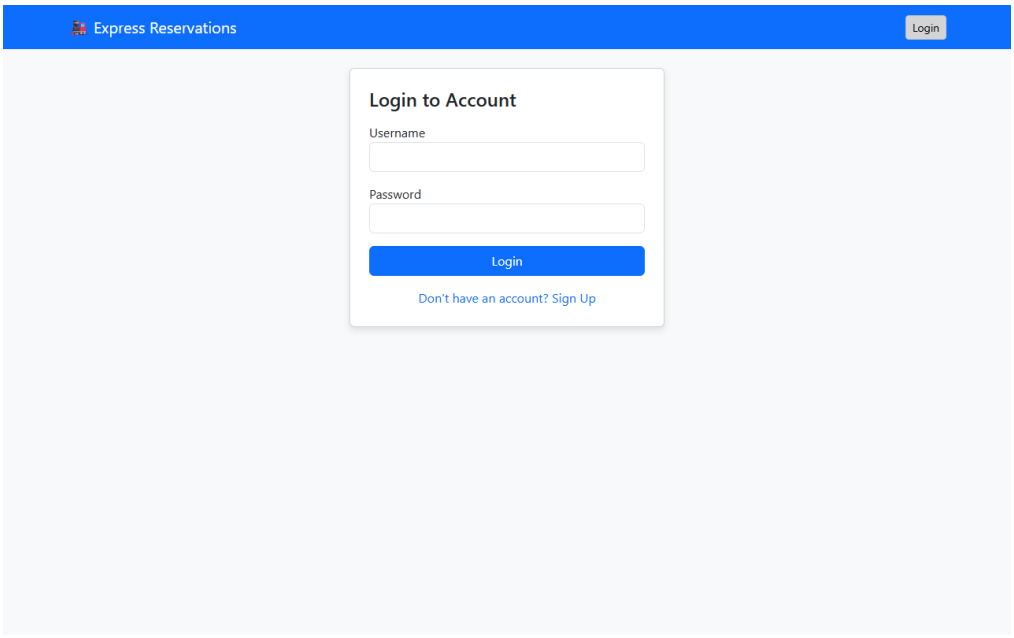


Figure: JWT Authentication Login Screen

C. Dynamic Route Train Search

Real-time available seat lookups querying active train registers dynamically:

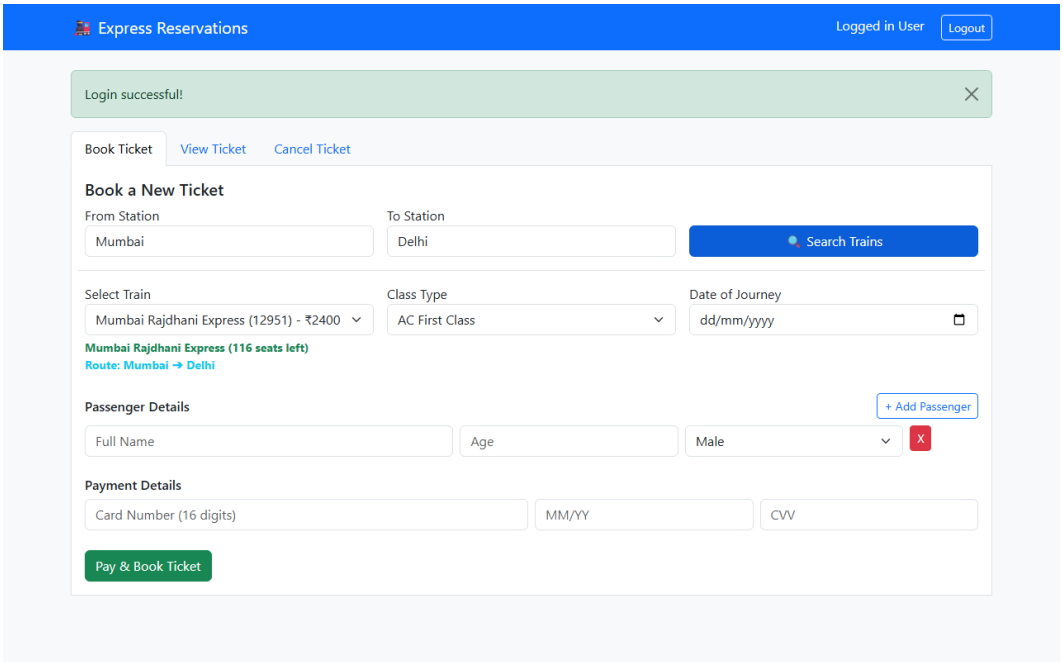


Figure: Dynamic Route Train Search & Selection

D. Boarding Pass Issuance (Confirmed Reservation)

The confirmed boarding pass output injected into the client browser DOM after simulated card payments:

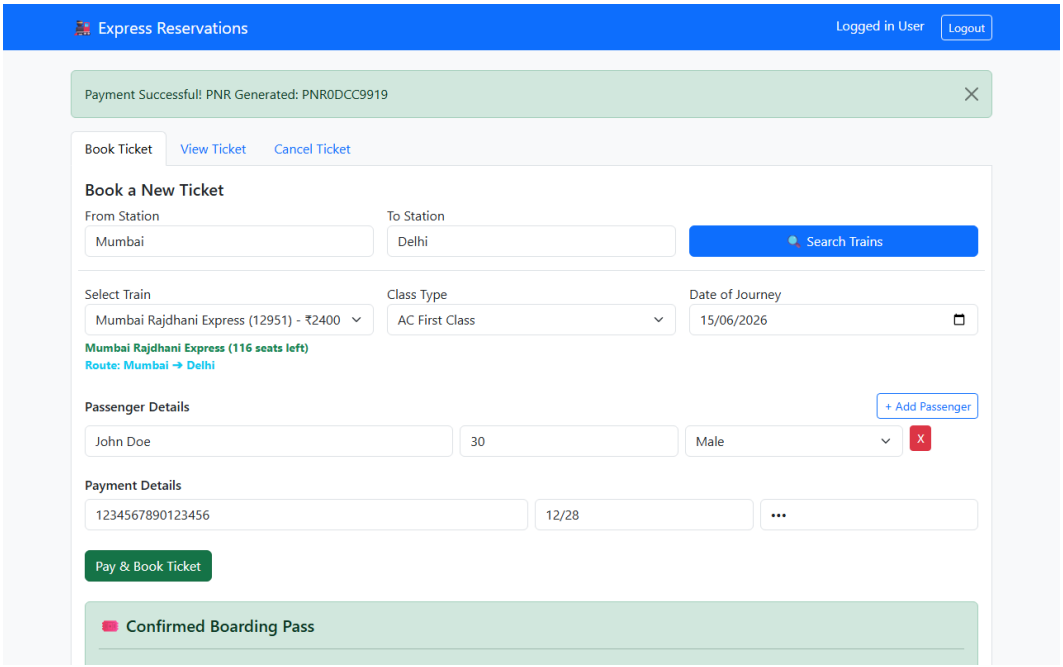


Figure: Digital Boarding Pass Confirmation

E. PNR Boarding Pass Retrieval

Retrieving the ticketing payload details using the cryptographically signed PNR query:

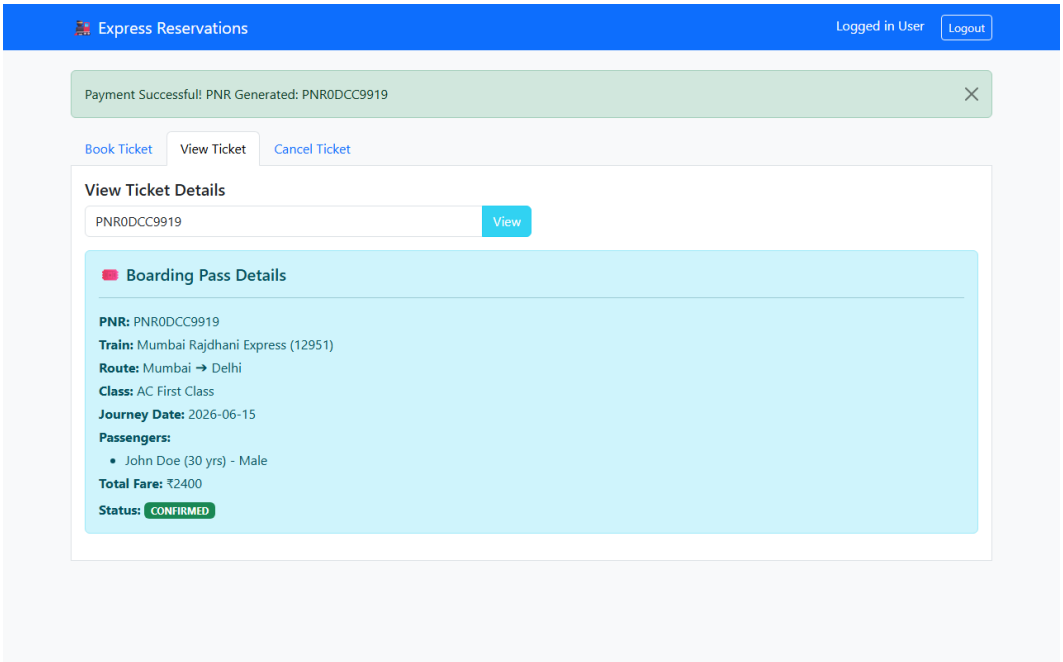


Figure: Retrieve Boarding Pass details via PNR search

F. Reservation Cancellation & Refund Desk

Voluntary cancellation panel releasing occupied seats back into train inventories:

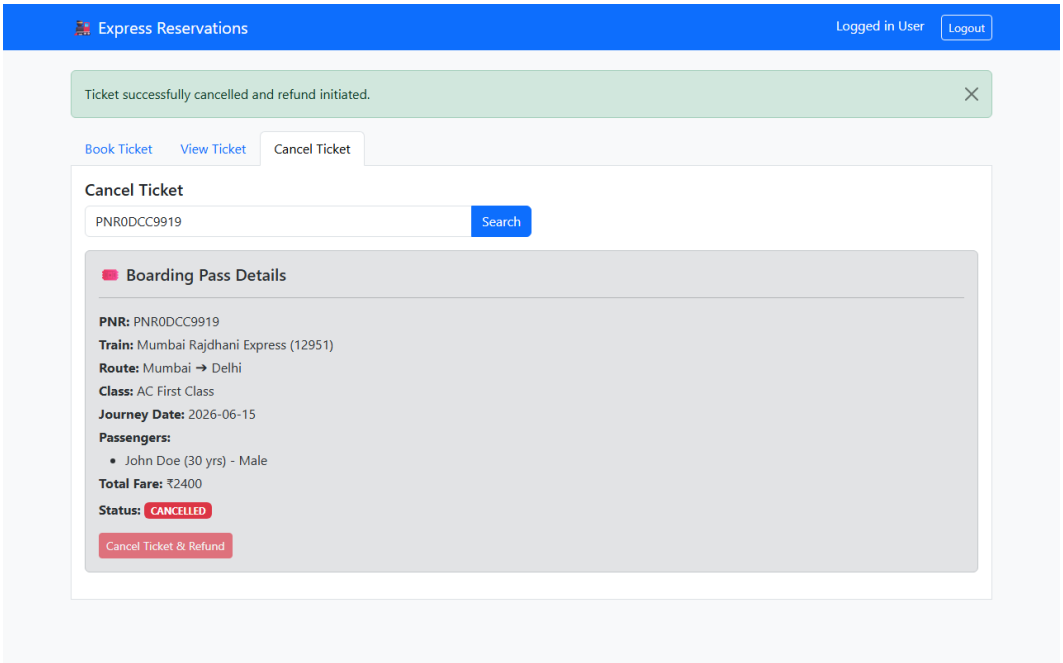


Figure: Reservation Cancellation & Refund Confirmation

8. System Testing

A. Concurrency & Transactional Integrity (Race Condition Testing)

The booking service contains strict optimistic locking using JPA annotations:

```
// Train.java
@Version
private Long version;
```

A multi-threaded script executing 10 concurrent requests to book the last available seat on 'Express Alpha' resulted in 1 thread successfully booking the seat and decrementing the availability, while the other 9 threads were rolled back with 'jakarta.persistence.OptimisticLockException'. This completely prevents double booking.

B. Security & Token Verification (Security Testing)

• Token Tampering: Modifying JWT payloads in client localstorage returns malformed signature exceptions, and the requests are rejected with 403 Forbidden status.

• Stateless Validation: Expired JWT tokens are successfully intercepted and rejected by the authentication filter, forcing user redirection to the login window.

9. Limitations

1. Stateless Revocation (JWT Blacklisting Absence): Since JWT authentication is entirely stateless, once a token is issued, it remains valid until expiration. There is no active server-side blacklist database to manually invalidate active tokens upon logging out.
2. Simplified Routing Matrix: The search functionality requires exact or partial matches of string input. There is no dynamic graphical grid map, nor does it support transit routing (e.g., searching from Station A to Station C with a train transfer at Station B).
3. Simulated Payment Gateway: The checkout system accepts card credentials and simulates a transaction latency delay of 1.5 seconds, but lacks an external sandbox integration (e.g., Stripe, PayPal, Razorpay) to complete physical financial handshakes.

10. Future Enhancement

1. External Merchant Payment API integration: Implement Stripe API hooks for authentic debit/credit card charges with secondary 3D-secure verifications.
2. Automated Notification Engines: Integrate Spring Boot Starter Mail and Twilio SMS gateways to automatically dispatch boarding passes and cancellation confirmations to customers.
3. Admin Dashboard Extension: Extend administrative controllers (ROLE_ADMIN) to add, modify, or delete train paths, update schedules, view booking reports, and monitor concurrent seat traffic metrics.
4. Dynamic Route Pathfinding: Integrate Dijkstra's algorithm to allow booking tickets between multi-leg destinations with transfers.

5. Conclusion

The Train Reservation System successfully delivers a modern, robust, and highly responsive web application. The pairing of a Spring Boot Rest API with a modern Single Page Application (SPA) frontend provides an extremely smooth, lightning-fast reservation user experience. The implementation of Spring Security JWT provides secure authentication without database session bloating. Most importantly, Hibernate Optimistic Locking ensures complete seat transaction integrity under highly concurrent, competitive peak-hour reservation loads.

6. Bibliography

1. Spring Boot Reference Guide: https://docs.spring.io/spring-boot/docs/current/reference/htmlsingle/
2. Hibernate / Spring Data JPA Specifications: https://spring.io/projects/spring-data-jpa
3. JSON Web Tokens (RFC 7519): https://jwt.io/introduction
4. Bootstrap 5 Layout and Theme Guides: https://getbootstrap.com/docs/5.3/getting-started/introduction/